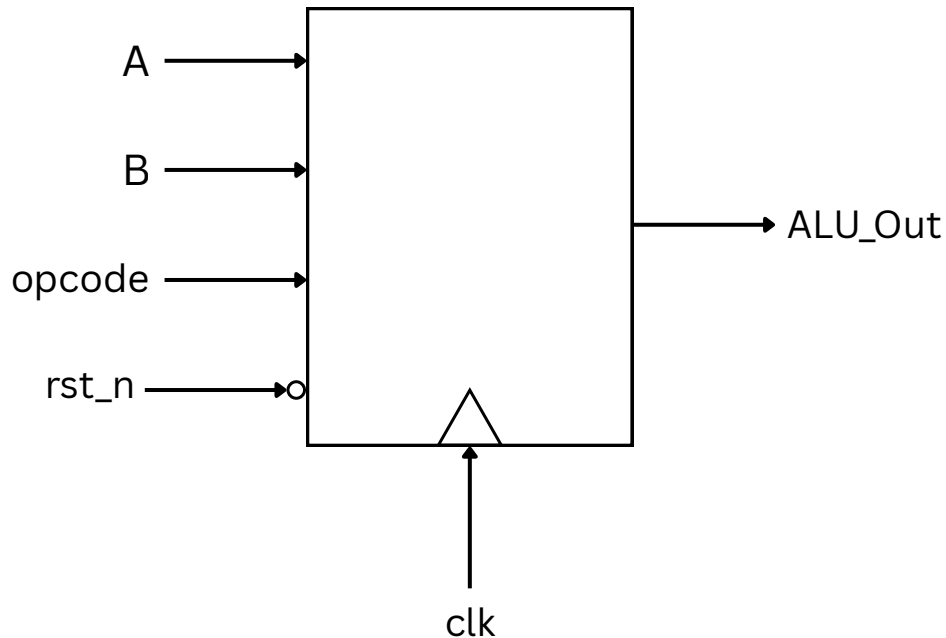


Verification Plan for ALU



- ALU with 4-bit operands and a 5-bit output
- Reset signal (asynchronous, active low)
- ALU_Out cleared to 0 whenever reset is asserted, regardless of clock
- On a positive clock edge, ALU_Out is loaded with $A + B$ when opcode = 00 (Addition)
- ALU_Out is loaded with $A - B$ when opcode = 01 (Subtraction)
- ALU_Out is loaded with $A \text{ AND } B$ (bitwise) when opcode = 10
- ALU_Out is loaded with $A \text{ OR } B$ (bitwise) when opcode = 11
- All operations are synchronous — ALU_Out only updates on the clock edge, never combinationally
- A and B are 4-bit inputs; opcode is a 2-bit input selecting the operation
- ALU_Out is 5 bits wide, so addition captures the carry-out (bit 4 set when $A + B$ exceeds 15) and subtraction captures the borrow (bit 4 set when $A - B$ goes negative, shown as the two's-complement wraparound)
- AND/OR results only ever use the lower 4 bits — bit 4 is always 0 for those two operations

Signals

1. RST → Asynchronous active low "ALU_Out must be Zero"
2. A, B (4-bit inputs) → Valid or Invalid
3. opcode → - @ Posedge CLK "Selects operation, no priority (single 2-bit select, not competing signals)"
 - Set to 00 → ALU_Out loaded with $A + B$ (Addition)
 - Set to 01 → ALU_Out loaded with $A - B$ (Subtraction)
 - Set to 10 → ALU_Out loaded with $A \text{ AND } B$ (bitwise)
 - Set to 11 → ALU_Out loaded with $A \text{ OR } B$ (bitwise)
4. ALU_Out (5-bit output) → Bit 4 reflects carry-out on Addition or borrow on Subtraction; always 0 on AND/OR

Test Scenarios

Random Sequence → fully randomized A, B, opcode for 30 transactions

Goal : exercising all four operations (Add, Subtract, AND, OR) and both carry-out/borrow cases under unconstrained random stimulus, relying on repetition to hit each opcode and edge case with reasonable probability